

Load Balancers

Task 1

In the AWS Management Console, launch two EC2 instances running a simple web server (use Amazon Linux 2 and install httpd or nginx), and verify both are accessible via their public IPs.

Steps followed:

1. Logged in to the AWS Management Console and opened the EC2 Dashboard.
2. Clicked "Launch Instance", named it "web-server-1", selected "Amazon Linux 2 AMI" and instance type "t2.micro" (Free Tier eligible).
3. Under "Advanced details", added the following script in the "User data" field to automatically install and start a web server:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "Hello from web-server-1" > /var/www/html/index.html
```

4. Selected the existing key pair, allowed HTTP (port 80) and SSH (port 22) in the security group, and launched the instance.
5. Repeated the same steps to launch a second instance named "web-server-2", using a similar script that wrote "Hello from web-server-2" to its index.html, so the two servers could be told apart.
6. Once both instances showed "Running" and passed status checks, copied their Public IPv4 addresses from the EC2 console.
7. Opened each Public IP in a browser and confirmed that both web-server-1 and web-server-2 responded correctly with their own "Hello from..." message, verifying that both were accessible individually.

Task 2

Set up an Application Load Balancer (ALB) in AWS that distributes incoming HTTP traffic across your two EC2 instances, and test by accessing the ALB's DNS name in your browser to see traffic served from both servers.

Steps followed:

1. In the EC2 Dashboard, went to "Target Groups" under Load Balancing and clicked "Create target group".
2. Chose target type "Instances", named it "web-servers-tg", set the protocol to HTTP on port 80, and selected the same VPC as the EC2 instances.
3. On the next screen, registered both web-server-1 and web-server-2 as targets and clicked "Create target group".
4. Went to "Load Balancers" and clicked "Create load balancer", then chose "Application Load Balancer".

5. Named it "web-app-alb", set it as internet-facing, selected at least two Availability Zones/subnets, and attached a security group allowing inbound HTTP traffic.
6. Added a listener for HTTP on port 80 and forwarded it to the "web-servers-tg" target group created earlier, then clicked "Create load balancer".
7. Waited for the load balancer's state to become "Active" and copied its DNS name from the console.
8. Opened the ALB's DNS name in a browser multiple times/refreshed repeatedly, and confirmed that responses alternated between "Hello from web-server-1" and "Hello from web-server-2", proving that traffic was being distributed across both instances.

Task 3

Change the health check settings of your ALB to use a custom path (e.g., /health) and verify that the load balancer only routes traffic to healthy instances. Hint: Create a /health endpoint on your web server that returns a 200 OK response.

Steps followed:

1. Connected to each EC2 instance using SSH and created a simple health check file with the command:

```
echo "OK" | sudo tee /var/www/html/health
```

2. Confirmed that visiting <Public-IP>/health in a browser returned "OK" with an HTTP 200 status on both instances.
3. In the EC2 console, opened "Target Groups", selected "web-servers-tg", and went to the "Health checks" tab, then clicked "Edit".
4. Changed the health check path from the default "/" to "/health", kept the protocol as HTTP, and saved the changes.
5. Waited a couple of minutes and checked the "Targets" tab of the target group, where both web-server-1 and web-server-2 showed the status "Healthy", confirming they were correctly responding on the new /health path.
6. To verify unhealthy targets get excluded, temporarily renamed the /health file on web-server-1 so the path returned a 404 error, and observed that its status changed to "Unhealthy" in the target group and the ALB stopped sending traffic to it, routing all requests to web-server-2 only. Restoring the /health file brought web-server-1 back to "Healthy" and traffic distribution resumed.

Task 4

Simulate a server failure by stopping one of your EC2 instances and observe how the ALB handles requests — describe what happens and how this demonstrates high availability.

Observation:

To simulate a failure, web-server-1 was stopped from the EC2 console while continuously refreshing the ALB's DNS name in the browser. For the first few requests immediately after stopping the instance, some requests

briefly failed or slowed down until the health check detected the instance was no longer responding. Within about 1-2 health check cycles, the target group marked web-server-1 as "Unhealthy" and automatically stopped routing any traffic to it. After that point, every single request to the ALB's DNS name was served only by web-server-2, with a "Hello from web-server-2" response, and no errors were shown to the user.

This demonstrates high availability because the failure of one server did not bring down the whole application — the load balancer automatically detected the unhealthy instance and rerouted all traffic to the remaining healthy instance, keeping the website accessible to users. When web-server-1 was started again, it eventually passed its health checks and was automatically added back into rotation, showing that the system can recover without any manual intervention on the load balancer itself.

Task 5

Compare Application Load Balancer, Network Load Balancer, and Classic Load Balancer in a table with at least 3 points each, focusing on supported protocols, use cases, and features.

Point	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Supported Protocols	Works at Layer 7 (application layer); supports HTTP and HTTPS.	Works at Layer 4 (transport layer); supports TCP, UDP, and TLS.	Works at both Layer 4 and Layer 7; supports basic HTTP, HTTPS, and TCP.
Use Cases	Best for modern web applications and microservices that need content-based routing, such as routing by URL path or hostname.	Best for use cases needing extremely high performance and low latency, such as gaming or real-time applications.	Best for simple, legacy applications built on the older EC2-Classic network, mostly used for basic load balancing needs.
Key Features	Supports advanced routing rules, path-based and host-based routing, and native support for containers.	Can handle millions of requests per second while preserving the source IP, and supports static IP addresses.	Offers only basic load balancing features and lacks the advanced routing capabilities of ALB and NLB.
Recommended Today	Recommended for almost all modern HTTP/HTTPS-based web and mobile applications.	Recommended when raw performance, low latency, or non-HTTP protocols like TCP/UDP are required.	Considered legacy; AWS recommends migrating to ALB or NLB for new applications.